scientific reports

Check for updates

OPEN

Enhanced intrusion detection system IoT network security model by feed forward neural network and machine learning

Abdullah Mujawib Alashjaee[1]✉ & Fahad Alqahtani[2]

The security of IoT networks has become a significant concern owing to the increasing count of cyber threats. Traditional Intrusion Detection Systems (IDS) struggle to detect sophisticated attacks in real-time due to resource constraints and evolving attack patterns. This study proposes a novel IDS that integrates deep learning (DL) and machine learning (ML) approaches to improve IoT security. The main objective is to develop a hybrid IDS combining Feed Forward Neural Networks (FFNN) and XGBoost to improve attack detection accuracy while minimizing computational overhead. The proposed methodology involves data preprocessing, feature selection utilizing Principal Component Analysis (PCA), and classification employing FFNN and XGBoost. The model is trained and evaluated on the CIC IoT 2023 dataset, which comprises real-time attack data, ensuring its practical relevance. The proposed model is estimated on the CIC IoT 2023 dataset, demonstrating superior accuracy (99%) compared to existing IDS techniques. This study provides valuable insights into improving IDS models for IoT security, addressing challenges such as dataset imbalance, feature selection, and classification accuracy. Results demonstrate that the hybrid FFNN-XGBoost model outperforms standalone FFNN and XGBoost classifiers, achieving an accuracy of 99%. Compared to existing IDS models, the proposed approach significantly enhances precision, recall, and F1-score, ensuring robust intrusion detection. This research contributes to IoT security by introducing a scalable and efficient hybrid IDS model. The findings offer a strong basis for future advancements in intrusion detection using DL and ML approaches.

Keywords Internet of things, Intrusion detection system, Feed forward neural networks, XGBoost, Cyber risks

In modern times, the IoT is defined as a network of interconnected computing de-vices that can share data via predefined protocols over the Internet. The IoT is a rela-tively new development that incorporates sensors and gadgets that are at the forefront of the domains of contemporary computing and communication. Currently, IoT sacri-fices security for its increased comfort and positive impact on human life. As an added bonus, hackers and intruders find this structure particularly appealing. Intrusion de-tection (ID) is the hardest thing to do in an IoT setting.

Nowadays, most computer networks implement IDS to monitor user behavior and automatically anticipate attacks by collecting and analyzing data[1]. Using cryptographic approaches for IoT security causes a lot of communication and processing overhead since IoT devices are resource constrained. The development and implementation of IDS may resolve this problem. Consequently, IoT contexts extensively employ IDS to detect imposters and ensure secure IoT communication[2].

In an IoT network, an IDS main function is to identify along with report any devi-ation from the expected behaviour of the system. When something goes wrong with the system, anomaly-based IDS will raise alarms because they are constantly checking the normal profile. It generates a lot of false alerts but works well against unknown threats. Owing to the developments in ML, a number of solutions have emerged; these solutions are able to continuously learn as well as adapt to new environments[3]. Over the last several years, many analytical approaches have been suggested for visualizing and pa-rameterizing neural networks' loss functions in the

[1]Department of Computer Sciences, College of Science, Northern Border University, Arar, Kingdom of Saudi Arabia. [2]Department of Computer Science, Prince Sattam Bin Abdulaziz University, Al Kharj, Riyadh, Kingdom of Saudi Arabia. ✉email: abdullah.alashjaee@nbu.edu.sa

Content courtesy of Springer Nature, terms of use apply. Rights reserved

space indicated by their weights. Many sophisticated network topologies employ feed-forward neural networks (FFNNs) as a classifier to tackle different kinds of large-scale prediction and classification issues. Most intricate network designs include the feed-forward layer[4].

The IoT may be genuinely omnipresent without actively seeking knowledge and data patterns, owing to ML, which enables it to glean hidden insights from the treasure trove of sensed data. Without widespread use of ML, the IoT will not be able to meet the demands of governments, organizations, as well as individuals in the future. In order to automate decision-making using intelligent approaches that resemble human deci-sion-making, the main objective of the IoT is to sense what's going on. Only by applying ML to its gadgets the IoT become truly pervasive. It is possible that AI with human-like intelligence is based on ML's pattern-finding tendency. This can learn more about the world by generalizing these patterns into useful insights as well as trends[5]. The ML real goal in the IoT is to achieve full automation by increasing learning, which in turn ena-bles intelligence through smarter things. Prior art IDS make the assumption that all IoT devices have the similar pattern of features as well as packet forms. In contrast, IoT devices differ in several ways. These include their computing capacity, the features they can generate, and the hardware features and functionality they provide[6].

A vast array of networked objects, such as public facilities, wearable technology, drones, etc., make up the IoT due to the technology's many potential uses. There are very serious security concerns, despite the fact that technology greatly improves the quality of life. Due to its massive size and intricate nature, the IoT network is notoriously dif-ficult to administer. When it comes to security flaws, IoT devices are easier targets than conventional networks[7]. One of the primary security mechanisms included in modern networks is IDS, which are able to detect attacks, illegal incursions, and negative activities inside networks. When an IDS detects unusual or out-of-the-ordinary behaviour on a host or network, it will notify the system administrator. While much work has been done, the IoT and dispersed networks are changing things. Errors are more likely to occur in more complicated networks. Insiders take benefit of their authorized access to the network to launch attacks, leaving little suspicious evidence behind. New attacks are continually being devised. Intelligent, data-driven systems will soon replace the older, more knowledge-based IDS. Instead of comparing traffic to a fixed signatures set as well as patterns, these systems should be able to study from data, see statistical regularities along with patterns, and spot intrusions that don't fit with normal traffic[8]. IoT security faces significant issues, including the inability of existing IDS to notice evolving cyber threats, resource constraints of IoT devices limiting security implementations, and increasing attacks complexity. Existing solutions struggle with real-time detection, scalability, and treatment of high-dimensional network traffic. Further the limitations with existing approaches include working with huge dataset, real-time and accuracy issues. To address with these limitations this research, works with the following contributions.

- This work designs and implements a novel hybrid intrusion detection framework that integrates an FFNN for deep feature extraction with XGBoost for high-precision classification, leveraging the strengths of both DL and gradient boosting.
- A two-step resampling strategy is employed with random oversampling of minority attack classes and under-sampling of majority classes. This mitigates dataset imbalance and reduces bias, improving detection fairness across classes.
- The proposed FFNN combines dropout layers and L2 regularization to enhance generalization when handling complex IoT traffic patterns.
- Using the CIC IoT 2023 dataset, good binary and multiclass detection performance is demonstrated and compared to recent state-of-the-art IDS models, supported by detailed metric-based comparisons.
- The proposed framework is computationally efficient and can be adapted for deployment in diverse IoT network environments.

The remaining section of the work is structured as follows. Section 2 lists the recent works related to the topic giving details about its pros and cons. Section 3 delivers a comprehensive proposed system description. In Sect. 4, the results are discussed. Section 5 concludes the study.

Literature review

This section examines some of the most recent IDS techniques using various methods. Table 1 summarizes the limitations as well as advantages of the previous methods.

Muhammad Sajid et al.[9] proposed a method that used CNNs as well as Extreme Gradient Boosting (XGBoost) to do feature extraction and then used Long Short-Term Memory (LSTM) networks for classification. Recent developments in cloud computing, the IoT, and automotive networks have led to a rise in the data quantity sent via communication infrastructures. As communication technology advances, network systems communicate varied and heterogeneous data in distributed settings. The reliability and security of the information provided by network security systems are crucial to the communications and everyday interactions that rely on these networks. Mean-while, malicious actors are making more noise about their efforts to compromise net-work infrastructure. An IDS is crucial because new technologies bring with them new forms of attack and security flaws. To overcome these restrictions, they have built a hybrid model for ID that combines DL as well as ML approaches.

Louai A. Maghrabi[10] suggested an Automated Network ID(NID) using the UNSW-NB15 dataset obtained from Kaggle and a well-known ML model called RF. NID systems notify system administrators of security breaches. An autonomous NID may better protect IoT networks against a variety of threats. Several ID methods exist; however, it is challenging to use them due to the dataset's performance issues and class imbalance. An improvement in ID accuracy is necessary to raise detection rates while decreasing false alarms. This approach was used in an effort to accomplish this.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Paper and Author	Method	Advantages	Limitations
Muhammad Sajid et al.[9]	DL and ML	Better Performance	Complexity in computation. Detection speed is quite less
Louai A. Maghrabi[10]	ML	Provides higher accuracy	This model does not develop real time Random Forest (RF) model that detects IoT Networks intrusions
Priyanka Singh et al.[11]	FFDN	classification accuracy is obtained	Insufficient to efficiently detect malware when trained using a large dataset
Ayoob Almotairi et al.[12]	ML	Capture valuable information and harness the strengths of individual models	Does not address the unique challenges posed by various types of IoT environments.
Jing Li et al.[13]	ML	Cost-effective and lightweight model capable of classifying attacks in IoT scenarios.	Do not apply on real time analysis on practical IoT scenarios.
Hossein Asgharzadeh et al.[14]	Convolutional Neural Network (CNN)	Better detects anamolies	Computational complexity
Alejandro Domínguez Campos et al.[15]	ML	simplicity, reproducibility and portability	Does not test the models against real malware in order to determine if the system is capable of successfully detecting it as malicious behavior
Saheed et al.[16]	Genetic Algorithm + Attention Mechanism + LSTM	Lightweight detection for multivariate IoT sensor data; high detection rate	Requires fine-tuning for different industrial IoT environments
Saheed et al.[17]	Hybrid GA for feature selection + optimized LSTM	Improved detection accuracy in IoT edge networks	Computational overhead may be high for resource-limited devices
Saheed et al.[18]	Deep autoencoder for feature extraction, CNN + LSTM for classification	Effective for ICS anomaly detection; handles temporal + spatial features	Limited generalization outside industrial control systems
Saheed et al.[19]	Explainable ensemble transfer learning	Accurate zero-day botnet detection; interpretable results	Model complexity increases deployment time in real-time IoV
Nazir et al.[20]	Hybrid CNN–BiLSTM with feature fusion	Achieves high detection accuracy and robustness in heterogeneous IoT traffic	Requires large computational resources for training

Table 1. Summary of previous works their methods, advantages along with limitations.

Priyanka Singh et al.[11] put out a strategy for using feed-forward deep neural network (FFDNN) hidden layers to extract deep features. The extraction of deep characteristics from hidden layers signifies a dataset with improved generalizability for malware detection. They used the Gaussian error linear unit (GeLU) activation function to pass the deep feature from one hidden layer to the next. To train FFDNN, this work uses the GeLU activation function, which considers the combined deep features of all hidden layers along with the deep characteristics of individual layers. They train and test the ML classifiers in a similar manner, using both deep features from individual layers and concatenated ones. They also examined the highly effective ML classifiers, including RF, k-nearest neighbour (k-NN), trees as well as support vector machine (SVM). Using a huge dataset that is statistically significant, they show how well their model performs.

Ayoob Almotairi et al.[12] created a stack classifier model for IoT data using heterogeneous ML. This model solved the problem of finding intrusions in the IoT, which can be hard to do. In order to study and improve critical classification metrics for ID of IoT data, the model uses ensemble modelling and feature selection. This method con-sists of two main parts: building an ensemble model using several classic ML models and using the K-Best method to select features (i.e., extracting the top fifteen most significant ones). By bringing these parts together, it may improve classification performance by using data from chosen characteristics and by combining the power of distinct models.

Jing Li et al.[13] did a study to see how ML-based attack categorization frameworks compare selection for IoT network ID as well as feature extraction. It uses binary and multiclass classification to examine performance metrics on the Network TONNE-IoT dataset, which contains heterogeneous IoT devices, such as accuracy, f1-score, runtime, etc. Since there are fewer characteristics to work with in feature extraction, it outperforms feature selection in terms of detection performance. Furthermore, compared to selection, feature extraction exhibits less feature reduction and is less susceptible to variations in the quantity of characteristics. In contrast to its alternative, feature selection reduces the amount of time needed for inference along with model training. In addition, when the number of characteristics changes, there is greater room to enhance the accuracy of selection rather than extraction.

Hossein Asgharzadeh et al.[14] developed FECNNIoT, an algorithm for IoT anomaly detection that utilizes feature extraction with CNN. They also developed BMEGTO, a binary multi-objective optimizer for Gorilla troops, to achieve feature se-lection. Finally, the introduction of a hybrid approach, CNN-BMEGTO-KNN, combines FECNNIoT with BMEGTO along with KNN algorithm-based classification techniques. The next stage was to apply it to TON-IoT as well as NSL-KDD datasets.

Alejandro Domínguez Campos et al.[15] demonstrated how to analyze an IoT network in order to identify threats by using side-channel methods that track the de-vices' power consumption. They demonstrate the use of a ML-driven monitoring system to detect intrusions while maintaining the regular operation of the devices. Several scenarios pass the tests, including custom-made datasets, models identifying novel at-tacks, and real-time testing. As mentioned in Table 1 though the existing works have several advantages such as better performance in terms of classification accuracy some limitations are identified.

Despite numerous IDS techniques, existing models face challenges including high computational complexity, low detection accuracy, and inefficiency in working with real-time attacks. Most existing works do not effectively

Content courtesy of Springer Nature, terms of use apply. Rights reserved

integrate DL and ML for improved performance. Furthermore, class imbalance issues persist unaddressed, leading to biased detection. This research presents a hybrid model that overcomes these gaps by leveraging FFNN and XGBoost, attaining superior accuracy and real-time adaptability.

Proposed methodology

To discover as well as classify possible vulnerabilities along with attacks inside IoT networks, this section proposes a hybrid framework for DL and ML-based ID. This section gives the proposed hybrid framework, which will be followed by an exhaustive assessment and an analysis of the major contributions of each stage. The next section offers a comprehensive explanation of the software, libraries, and ID dataset employed in this study. The proposed system can extract packets from the data generated by the primary IoT infrastructure as well as detect intrusions after appropriate training. To gather the traffic flow from IoT devices, the proposed framework makes use of a net-work capture technique from an IoT network. After that, it uses the data to construct a model that acts like an IoT smart home. This trains the proposed system using the CIC IoT 2023 dataset, which contains packets from several networks. To find the IoT net-works anomalies, the proposed architecture shown in Fig. 1 uses a hybrid approach. Data preprocessing, feature selection, and data splitting are the three processes that make up the proposed model workflow.

The proposed hybrid model follows these key steps. With data preprocessing, the CIC IoT 2023 dataset undergoes data cleaning, feature scaling, and encoding. Principal Component Analysis (PCA)is utilized to choose the most important features. The FFNN extracts deep features from network traffic data. The extracted features are passed to XGBoost for final classification. Hyperparameters are tuned using RandomizedSearchCV to improve performance.

Dataset

The CIC IoT Dataset 2023, a relatively new dataset, serves as the foundation for the development of IoT security analytics tools[21]. This carried out seven types of attacks on 105 IoT devices: DDoS, Denial of Service (DDoS), Reconnaissance, Spoofing, Mirai, Web-based, and Brute Force. Malicious IoT devices launch these attacks on other IoT devices. By overpowering a single source with traffic, denial-of-service attacks impair services. Web-based attacks employ methods like XSS to target web applications as well as SQL injection. Brute force attacks attempt to gain unauthorized access via a series of trials. A spoofing attack is one that alters network traffic or pretends to be an entity. Last but not least, Mirai attacks use techniques such as UDPPlain along with GREIP Flood attacks, that target IoT devices. It also provides a comprehensive summary of network attacks, where the frequency of attacks is represented by the number of rows, as well as the types of attacks themselves are classified[22].

Data preprocessing

Before being passed on to the proposed hybrid model, the dataset features undergo cleaning, feature scaling, and encoding exercises. To work with class imbalance, oversampling of minority attack classes and undersampling of majority classes are employed in this research. To work with class imbalance in the CIC IoT 2023 dataset, a two-step resampling approach is applied. This includes random oversampling of minority attack classes to increase

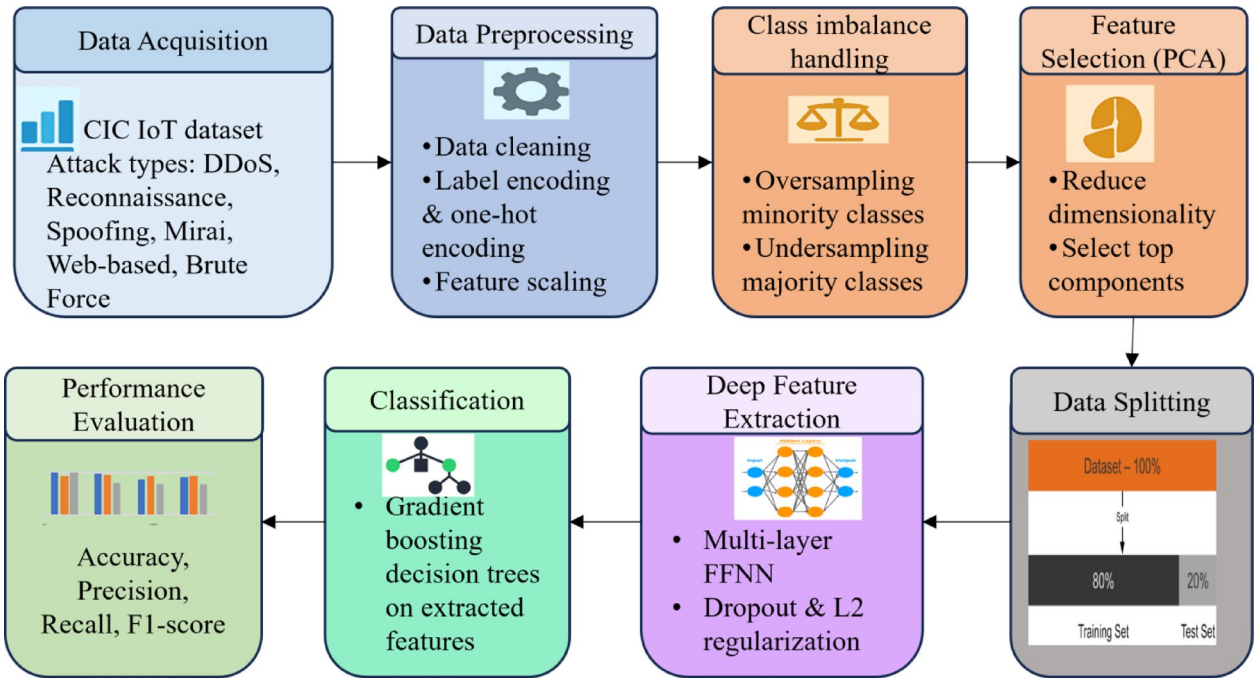

Fig. 1. Workflow of the proposed model.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

their sample count. Also, random undersampling of majority classes is done to reduce their dominance in the dataset. This ensures that during training each class has approximately equal representation, which reduces classification bias. Formally, for a class C_i with n_i samples and the maximum class size n_{max}, oversampling replicates samples until $n_i \approx n_{max}$. While undersampling reduces classes with $n_j > n_{target}$ to n_{target} samples. The resampling applies only to the training data, preserving the natural distribution in the test set for realistic evaluation. This ensures that the model was trained on a balanced dataset, reducing bias and thus improving classification accuracy.

Data cleaning

Data cleaning inspects for errors in the dataset, incorrect organization, repetition, or missing information and fixing them. The data cleaning process to be accurate and consistent every time, it needs to set up a framework. This study makes use of a cleaned CIC IOT 2023 dataset, which has superfluous packets removed and missing or infinite values removed.

Categorical encoding

Label encoding is the process of transforming numerical data into categorical data. To make data processing by algorithms more efficient, it is necessary to numerically assign values to each separate categorical variable. At this stage, it uses label encoding to give the features numerical values that are appropriate. Using one-hot encoding to encode categorical variables as binary vectors is another way to convert categorical data to numerical data in Eq. (1).

With n distinct values $x_1, x_2, \ldots, x_n$, consider a discrete categorical random variable, represented by x. In the process of one-hot encoding, a particular number x_i is used as a vector v, with all elements of v being zero except for the i th element, which is given the value 1.

$$v_{ij} = \begin{cases} 1 & if\ x_i = x_j \\ 0 & otherwise \end{cases} \tag{1}$$

Feature scaling

Changing the series of values within a features collection to match a predetermined range is known as feature scaling. Use this strategy when a feature holds significant value but its impact on other features is minimal[23]. Examples of this include normalization and standardization[24]. To lessen the impact of substantial variances throughout the merging process, normalization is a crucial step. This is realized by optimizing and scaling all features to a normal series. This can uniformly express column values by using Min-Max normalization with a chosen range of [0, 1] in Eq. (2).

$$x_{norm} = \frac{x - min\,(x)}{max\,(x) - min\,(x)} \times\ (b - a) + a \tag{2}$$

Here is where the original value that has to be normalized is represented by x. The lowest column feature value is represented as $min\,(x)$. The biggest column feature value is $max\,(x)$. The normalised value is x_{norm}. a represents the minimum acceptable level of normalization and b represents the highest level of normalization possible.

Feature selection

Feature selection is to decrease the dimensionality of data while retaining its features. It aids in enhancing storage efficiency and lowering computational complexity. When developing classifiers for IDS, it is necessary to use low-dimensional features for multiclass as well as binary classification because of the power limitations of IoT devices. According to the data, this method has produced remarkable results. This study, use PCA to reduce the original dataset to a lower-dimensional subspace. Additionally, PCA is used for reducing dimensionality while preserving meaningful features. Unlike feature extraction, which creates new variables, PCA ensures that the most relevant attributes from the original dataset are retained. Also, the essential qualities of the data are maintained. PCA is a way of doing statistical analysis that tries to decrease the features count in a dataset by employing orthogonal arrangements of the new values, which show a lot of variation[25]. By not relying on any one other, the principal components remove connected features that add little value to the decision-making process. The primary operations in PCA include calculating eigen-vectors, eigen-values, cumulative percentage, standard deviation, and covariance[26]. The significance of the dataset dictates the selection of combinations. Linearly combine the original variables to create the principal components, encapsulating the extreme degree of variance in the dataset. Basically, PCA entails:

The matrix X, representing dataset, has dimensions m × n. In Y, every row signifies a separate data point, also every column denotes a different feature. Equation (3) shows the calculation of the mean vector, denoted as μ.

$$\mu\ = \frac{1}{m} \sum {}_{i=1}^{m} X_i \tag{3}$$

The mean-centered data matrix, represented by X_i, is obtained by subtracting the mean vector from every data point in Eq. (4).

$$X_c = X_i - \mu \tag{4}$$

Content courtesy of Springer Nature, terms of use apply. Rights reserved

The covariance matrix C is generated by finding the center of the mean covariance among the data's attributes in Eq. (5).

$$C = \frac{1}{m}\left(X_c^T.X_c\right) \tag{5}$$

An eigenvalue method decomposes the covariance matrix to yield eigenvalues as well as the eigenvectors. This refers to the matrix comprising the eigenvectors V as the matrix containing the eigenvalues λ in Eq. (6).

$$C = V\lambda V^T \tag{6}$$

The eigenvectors that represent each column in the v matrix have dimensions n × n. Also, the eigenvalues of λ are located on the diagonal of the diagonal matrix. To build the matrix, pick the K eigenvectors that match the v_k biggest eigenvalues. The vector P consists of K elements, represented by the symbols v_1 through v_k in Eq. (7).

$$P = [v_1, v_2, \ldots, v_k] \tag{7}$$

It is clear that each column represented by v_i represents a principal component. A method is used to decrease the data's dimensionality by projecting the mean-centered data into the K principal components.

$$X_{pca} = X_c P \tag{8}$$

Through correlation analysis, it can learn how dataset variables are related. Features with values higher than the threshold are excluded from the created correlation map, which aims to detect features with high levels of correlation. To build a model, it chooses features whose correlation is strong enough to surpass a certain threshold value of 0.95. This converts the $X_{resampled}$ data into X_{pca} using dimensionality reduction based on Eq. (8), where each row characterizes each column as well as a data point represents the principal component[27].

Data splitting

The feature selection step, split the processed data into two parts, training and testing. Both binary and multiclass classifications are possible with the processed data. When assessing a model's efficacy, it is common practice to split the dataset in half, with one half serving as a training set as well as the other half as a testing set. This uses the processed data to train the proposed hybrid FFNN with XGBoost model, as well as other existing models. The FFNN is trained using the categorical cross-entropy loss function for both binary and multiclass classification, chosen for its suitability in multi-class probability distribution learning. This function measures the divergence between predicted probabilities and actual class labels, guiding effective weight updates during backpropagation. This uses the test set to assess each model's accuracy. The model categorizes an 'Attack' along with 'Normal' class for multiclass along with binary classification issues.

Proposed hybrid detection model

The FFNN model has shown success in several applications, comprising recognition as well as classification. FFNN was chosen due to its ability to handle complex, high-dimensional IoT network traffic patterns. Its deep structure enables better feature extraction, while XGBoost enhances classification efficiency, making them a powerful combination for intrusion detection. The FFNN architecture demonstrates hyperparameter optimisation through the use of RandomizedSearchCV, the KerasClassifier wrapper as well as Scikit Learns. The hyperparameter optimization goal is to determine the optimal hyperparameters for a specific dataset. Four hyperparameters dictate the neural network architecture such as the hidden layers count, the dropout rate (to avoid overfitting), the L2 regularisation coefficient as well as the number of neurons in each hidden layer. To work with the overfitting issue, dropout layers ranging between 0.1 and 0.5 are applied after hidden layers, and L2 regularization is added with coefficients 0.001, 0.01, and 0.1. By these techniques, model variance is reduced and generalization is improved when dealing with complex IoT network traffic. This work uses an appropriate collection of hyperparameters to fine-tune the model, improving classification accuracy also reducing the danger of overfitting[28]. The dropout regularisation approach is employed after each hidden layer to prevent overfitting. Several specifications are available for hyperparameter selection, as indicated below in Table 2.

Here are the steps to generate the mathematical depiction of the proposed FFNN:

Hyperparameter	Value options
Hidden Layers	1, 2, 3
Neurons per Layer	64, 128, 256, 512
Dropout Rate	0.1, 0.2, 0.3, 0.4, 0.5
Activation Function	ReLU
Optimizer	Adam
Regularization	L2 (0.001, 0.01,0.1)

Table 2. Proposed hyperparameters.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

- It expresses the initial hidden layer output for a specific dataset, where x represents the input vector in Eq. (9).

$$h_1 = ReLU\left(W_1.x + b_1\right) \tag{9}$$

W_1 represents the weight matrix that joins the input and the initial hidden layers, b_1 stands for the bias vector, as well as ReLU stands for the activation function that applies the RELU. h_1 is the result of computing the initial hidden layer's output.

- Equation (10) signifies the neural network's output which is led to successive hidden layers, where i is the layer index.

$$h_i = ReLU\left(W_i.h_{i-1} + b_i\right) \tag{10}$$

- The output layer can be denoted numerically as follows in Eq. (11):

$$y = softmax\left(W_o.h_{num_hidden_layers} + b_o\right) \tag{11}$$

where b_o is the bias vector as well as W_o is the weight matrix. W_o links the output layer along with the final hidden layer.

- The dropout approach uses the masking procedure to set certain input units to zero during training. M_i, a binary mask with the same form as the relevant layer's output, may be used to indicate each layer masking process. In the training phase, it creates each output layer by multiplying the associated mask element-wise, as demonstrated in Eq. (12)

$$h_i = M_i.ReLU\left(W_i.h_{i-1} + b_i\right) \tag{12}$$

- This uses the L2 regularization loss function to improve weight minimization. It adjusts the weight matrices of each layer as per the loss function. Multiplying the L2 regularization coefficient by 0.5 yields the L2 regularization in Eq. (13).

$$L2_{reg} = 0.5.l2_{reg}.\left(\left|\left|W_l\right|\right|^2 + \left|\left|W_2\right|\right|^2 + \ldots + \left|\left|W_o\right|\right|^2\right) \tag{13}$$

where $|W_i|$ is the Frobenius norms sum of the weight matrices W_i and $l2_{reg}$ is the L2 regularisation term. To discover the optimal hyperparameters combination to identify intrusions in the dataset, it will randomly search through them. Using both the training as well as validation sets, the model goes through a series of iterations to discover the best set of hyperparameters that maximizes presentation with RandomizedSearchCV while minimizing processing time. The automatic proposed model's optimization as well as regularization parameters eliminates the need for saving time, energy along with manual tuning.

Figure 2 displays the FFNN architecture layer contribution. The XGBoost classifier takes the FFNN's output as input and stores it as features. The primary goal in devel-oping XGBoost was to maximize performance and speed using gradient-boosted deci-sion trees. Tianqi Chen initiated XGBoost, a method that applies boosting to machines, and numerous developers have since adopted it. The Distributed ML Community (DMLC) is the proud owner of this fantastic technology. For hardware boosting meth-ods, XGBoost, or eXtreme Gradient Boosting, aids in making full use of available memory and boosting capabilities. It is useful for improving algorithms, deploying them in computer settings along with fine-tuning models. Regularized Boosting, Stochastic Boosting as well as Gradient Boosting, are the three main gradient boosting algorithms that XGBoost can execute. The method excels in optimizing memory use and reducing computation time. It has the unique ability to perform boosting on additional data that is already in the trained model, is Sparse (i.e., it can handle missing values), and allows parallel structured tree building. Finally, the system analyzes the produced outcomes using several performance measures.

Results and discussions

This section offers a detailed description of the proposed hybrid DL models for ID, along with detailed specifications. Execution and evaluation of the proposed technique were carried out using the Jupyter Notebook program, an interactive Python-based integrated development environment (IDE) included in the Anaconda package. This used Keras, a Python-based DL framework, in the research. A Core i5 processor with 16 GB of RAM is the hardware used. It employs the open-source package Pandas and the Python 3.10.4 computer language for data analysis. Batch size, learning rate, also optimizer type is employed for loss function minimization. With optimization, it is possible to minimize the cost function with little effort and resources. After looking at all of the other options, this work chooses the ADAM optimizer because it can maximize the categorical cross-entropy loss for many classes. For training 80% data is used and testing gets the remaining 20% data.

Evaluation metrics

This work employs the common evaluation metrics including recall, precision, accuracy, along with F1 score. This work uses these metrics to scrutinize the proposed models. Accuracy, recall, precision, and F1-score were chosen since they give a comprehensive intrusion detection performance evaluation. Accuracy measures overall exactness, while precision and recall highlight the ability to classify threats effectively. F1-score balances precision along with recall, ensuring robustness in imbalanced datasets. The "accuracy" of a model is defined as

Content courtesy of Springer Nature, terms of use apply. Rights reserved

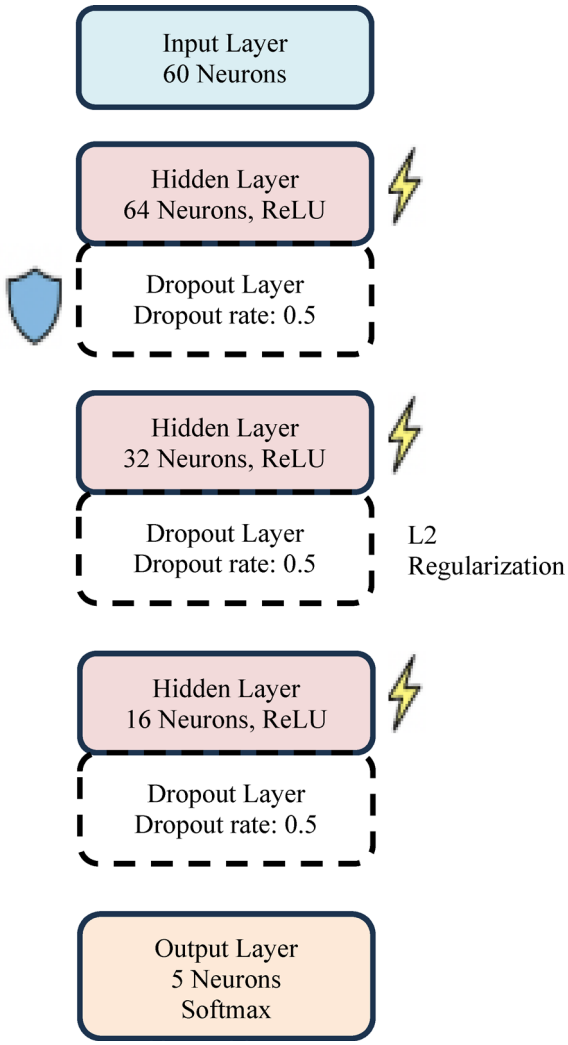

Fig. 2. Architecture of FFNN.

the precision with which its predictions meet the actual data. The accuracy rate represents the correctly classified cases percentages among all instances, encompassing both true positives and negatives. This utilizes Eq. (14) to perform the calculation.

$$Accuracy = (TP + TN)/(TP + TN + FP + FN) \tag{14}$$

- True Positive (TP) refers to the quantity of incidents properly identified as positive or threats.
- True Negative (TN) represents the cases count properly classified as negative, indicating they posed no threats.
- False Positive (FP): The number of occasions when non-threats were mistakenly identified as a threat.
- False Negative (FN): the total cases count when threats are wrongly labelled as non-threats or where the negative classification is applied erroneously.

Precision, mean the level of accuracy of positive predictions. This anticipated a total of positive occurrences, including both true and false positives. The precision formula can be obtained as follows in Eq. (15):

$$Prec = TP/(TP + FP) \tag{15}$$

Recall, often referred to as true positive rate, is the percentage of correctly anticipated positive events (true positives) to all positive events. Equation (16) provides the mathematical expression:

$$Rec = TP/(TP + FN) \tag{16}$$

The F1 score integrates recall and precision into a single measure. Consequently, it achieves equilibrium between the two metrics. When there are inconsistencies in the selections for precision and recall, averaging the two helps. Statistical analysis as well as ML often use this to assess the classification model efficacy. Equation (17) demonstrates the exact expression:

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Class / Specification	Binary	Multiple
Hidden layer neuron	64	512
Dropout layer size	0.1	0.1
Dropout Activation	Relu	Relu
Output layer size	2	8
Output activation	Softmax	Softmax

Table 3. FFNN specifications.

	Precision	Recall	F1-score	support
0	0.94	0.96	0.95	2433
1	0.99	0.99	0.99	10,096
Accuracy			0.99	10,340
Macro avg	0.97	0.98	0.97	10,340
Weighted avg	0.99	0.99	0.99	10,340

Table 4. Proposed results for binary class.

	Precision	Recall	F1-score	support
0	0.92	0.98	0.95	2433
1	0.99	0.99	0.99	75,261
Accuracy			0.99	10,340
Macro avg	0.96	0.84	0.88	10,340
Weighted avg	0.99	0.99	0.99	10,340

Table 5. Proposed results for multi-class.

Method / Parameter	Accuracy	Precision	Recall	F1-score
Proposed	99	97	98	97
FFNN	90.41	94.75	90.41	92.30
XGBoost	99.66	95.36	97.51	96.41
GA-mADAM-IIoT [16]	98.50	96.80	97.20	97
Modified GA + LSTM [17]	97.90	95.10	96.40	95.70

Table 6. Existing work comparison for binary class.

$$F1 - score = 2\left(prec * rec\right)/prec + rec \tag{17}$$

The unique F1 score's value emphasizes the mutual performance of recall along with precision. Table 3 summarizes the FFNN requirements.

Result analysis

In this section the results of the proposed hybrid model are done by using CIC IoT dataset 2023. Both the binary class and multiclass results are tabulated in Tables 4 and 5.

Table 4 shows the binary class performed well, with maximum precision of 0.97%, recall of 0.98%, accuracy of 0.99% and an F1-score of 0.97%.

Table 5 displays the multiclass model; the highest accuracy is 0.99%, precision is 0.96%, recall is 0.84%, as well as F1-score is 0.88%. When comparing both the binary and multiclass values the accuracy for both binary and multiclass is 0.99%. The binary class precision is 1.04% better than multiclass. Similarly, the binary class recall is 16.66% better than multiclass. Finally, the binary class precision is 10.22% better than multiclass. This concludes that though there is not big difference in accuracy value between the binary class and multiclass, the other performance parameters show that the binary class provides the better results than the multiclass dataset. This is since as the number of instances increase in some cases the performance falls.

Table 6 displays the comparison results for binary class. The proposed approach demonstrates a precision of 97%, recall of 98%, accuracy of 99%, and f1score of 97%. This study estimates the percentage difference between the proposed and the existing algorithms. This indicates that the proposed model outperforms FFNN by 2.37%

Content courtesy of Springer Nature, terms of use apply. Rights reserved

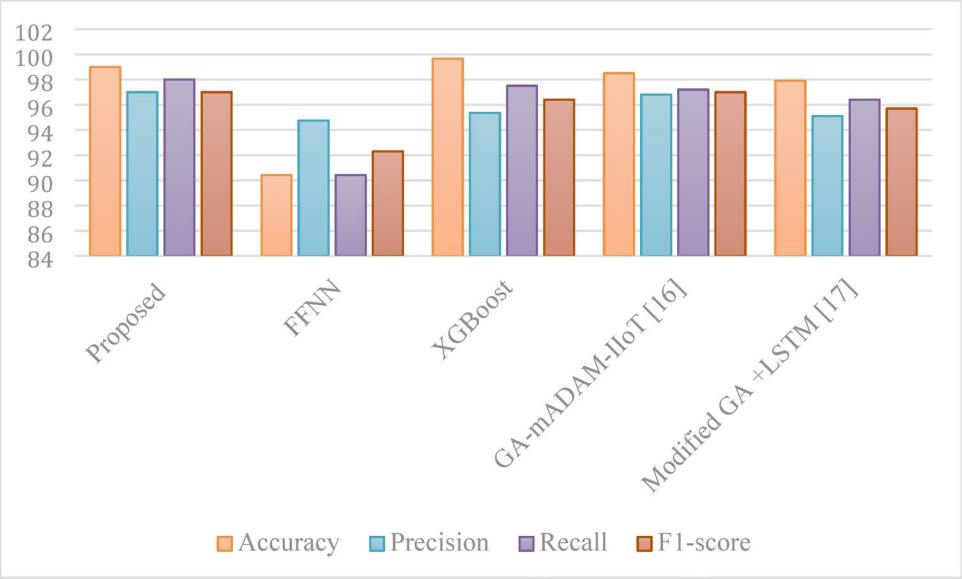

Fig. 3. Comparison graph of binary class.

Method / Parameter	Accuracy	Precision	Recall	F1-score
Proposed	99	96	84	88
FFNN	90.24	95.26	90.24	90.34
XGBoost	99.31	74.16	71.49	72.56
XAIEnsembleTL-IoV [19]	98.70	93.50	90.20	91.80
Hybrid CNN-BiLSTM [20]	98.90	94.80	92.10	93.40

Table 7. Comparison for multi-class.

and XGBoost by 1.71% in terms of precision. In terms of recall, the proposed model is 8.39% better than FFNN and 0.50% better than XGBoost. The proposed model's accuracy is 9.50% higher than FFNN and 0.66% higher than XGBoost. Finally, the proposed model's f1score is 5.09% better than FFNN and 0.61% better than XGBoost. When compared to GA-mADAM-IIoT, the proposed model achieves 0.21% higher precision, 0.82% higher recall, and 0.51% higher accuracy, with equivalent F1-score performance. Against the Modified GA + LSTM model, the proposed model improves precision, recall, accuracy, and F1-score by 2.00%, 1.66%, 1.12%, and 1.36%, respectively. Figure 3 shows a graph comparing the proposed binary class with existing techniques.

Table 7 displays the multi-class comparison results. The proposed approach demonstrates a precision of 96%, recall of 84%, accuracy of 99%, along with f1score of 88%. This study estimates the percentage difference between the proposed and existing algorithms. This indicates that the precision of the proposed model surpasses FFNN by 0.77% and surpasses XGBoost by 29.44%. In terms of recall, the proposed model is 7.42% better than FFNN and 17.49% better than XGBoost. The proposed model's accuracy is 9.70% higher than FFNN and 0.31% higher than XGBoost. Finally, the proposed model's f1score is 2.65% better than FFNN and 21.27% better than XGBoost. In multiclass classification, while the proposed method surpasses XAIEnsembleTL-IoV in precision and accuracy, it records slightly lower recall and F1-score, likely due to the complexity of multiclass distributions. A similar trend is observed when compared to Hybrid CNN–BiLSTM. The efficient performance of the proposed hybrid FFNN–XGBoost model indicates the deep feature extraction capabilities of FFNN. This model captures complex traffic patterns, combining the pros of XGBoost in handling high-dimensional, sparse data efficiently. This hybrid work enables better differentiation between subtle anomalies and benign traffic, which non-hybrid models struggle to achieve. Figure 4 shows a graph comparing the proposed multi-class with existing approaches.

For both binary as well as multiclass, Tables 5 and 6 show how the proposed models compare to the most up-to-date DL-based ID systems. It looked at different DL and ML models independently to see how successful the proposed hybrid model is. Both the proposed and existing algorithms' accuracy were the primary concerns. It employs the CIC IoT real-time dataset to identify network intrusions. The proposed hybrid model's accuracy for binary class is 99.78%, followed by FFNN (90.41%) and XGBoost (99.56%). The accuracy for multiclass using the proposed hybrid model is 99.66%, FFNN is 90.24%, and XGBoost is 99.31%. This work uses nearly eight classes for its multiclass analysis. Here, the fluctuation in the number of classes makes binary classification produce better results than multiclass classification. When deployed independently, the proposed hybrid model using both of these techniques outperformed the FFNN and XGBoost models. This combines the best features

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Fig. 4. Comparison graph of multi-class.

of FFNN and XGBoost to achieve this. While XGBoost excels in speed tests and has no trouble handling large datasets, FFNNs are better at handling complicated patterns of traffic in IoT networks. The proposed hybrid model also works well than the existing works when looking at additional metrics including recall, precision, as well as F1-score.

Conclusion and future work

This research enhances IoT network security by developing a robust and scalable IDS capable of addressing the evolving landscape of cyber threats. A hybrid FFNN–XGBoost model is proposed that combines the deep feature learning capabilities of FFNN with the classification precision of XGBoost. Through systematic preprocessing, including class imbalance handling and PCA-based feature selection, the model achieves high detection accuracy, precision, recall, and F1-scores on the CIC IoT 2023 dataset. The findings show that dropout layers and L2 regularization effectively reduce the overfitting issue, while the resampling approach ensures balanced learning across attack classes. Comparative analysis is done with recent existing IDS models, which confirms the proposed method effectiveness with both binary and multiclass datasets. The implications of this work are twofold: it provides a scalable and adaptable IDS framework for real-world IoT deployments, and it demonstrates that combining DL with boosting-based classifiers can yield significant performance gains. Future research will focus on integrating k-fold cross-validation and early stopping to further enhance generalization, as well as extending the model to handle encrypted IoT traffic and real-time detection environments.

Data availability

The data that support the findings of this study are openly available in CIC IoT Dataset 2023 (kaggle.com).

Received: 30 November 2024; Accepted: 11 September 2025
Published online: 15 October 2025

References

1. Saravanan, V. et al. IoT-based blockchain intrusion detection using optimized recurrent neural net-work. *Multimedia Tools and Applications*. **83**(11), 31505–31526 (2024).
2. Santhosh Kumar, S. V. N., Selvi, M. & Kannan, A. A comprehensive survey on machine learning-based intrusion detection systems for secure communication in internet of things. *Comput. Intell. Neurosci.* **1**, 8981988 (2023).
3. Ullah; Imtiaz; Qusay H. Mahmoud. An anomaly detection model for IoT networks based on flow and flag features using a feed-forward neural network. *In 2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, IEEE, 2022. 363–368.
4. Xue; Yu; Yiling Tong; Ferrante Neri. An ensemble of differential evolution and Adam for training feed-forward neural networks. *Inf. Sci.* **608**, 453–471 (2022).
5. Jamal, B. et al. Machine learning-enabled internet of things (iot): Data, applications, and industry perspective. *Electronics* **11**(17), 2676 (2022).
6. Khalid, A. et al. IoT intrusion detection using machine learning with a novel high performing feature selection method. *Appl. Scie.* **12**(10), 5015 (2022).
7. Jie, Y. et al. Internet of things intrusion detection system based on convolutional neural network. *Comput. Mater Contin.* **75**, 2119–2135 (2023).

Content courtesy of Springer Nature, terms of use apply. Rights reserved

8. Stefanos, T., Lagkas, T. & Rantos, K. Deep learning in IoT intrusion detection. *J. Network Syst. Manag.* **30**(1), 8 (2022).
9. Muhammad, S. et al. Enhancing intrusion detection: a hybrid machine and deep learning approach. *J. Cloud Comput.* **13**(1), 123 (2024).
10. Maghrabi; Louai, A. Automated Network Intrusion Detection for Internet of Things Security Enhancements. IEEE Access **2024**.
11. Priyanka, S. et al. Feed-forward deep neural network (FFDNN)-based deep features for static malware detection. *Int. J. Intell. Syst.* **1**, 9544481 (2023).
12. Almotairi, A., Atawneh, S. & Osama, A. Enhancing intrusion detection in IoT networks using machine learning-based feature selection and ensemble models. *Syst. Sci. Control Eng.* **12**(1), 2321381 (2024).
13. Jing, Li. et al. Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning. *J. Big Data* **11**(1), 36 (2024).
14. Asgharzadeh; Hossein; Ali Ghaffari; Mohammad Masdari; Farhad Soleimanian Gharehchopogh. An Intrusion Detection System on The Internet of Things Using Deep Learning and Multi-objective Enhanced Gorilla Troops Optimizer. *Journal of Bionic Engineering* **2024**, 1–27.
15. Campos; Alejandro Domínguez; Felipe Lemus-Prieto; José-Luis González-Sánchez; Andrés Caro Lindo. Intrusion detection on IoT environments through side-channel and Machine Learning techniques. *IEEE Access* **2024**.
16. Saheed, Y. K., Omole, A. I. & Sabit, M. O. GA-mADAM-IIoT: A new lightweight threats detection in the industrial IoT via genetic algorithm with attention mechanism and LSTM on multivariate time series sensor data. *Sensors Int.* **6**, 100297 (2025).
17. Saheed, Y., Kayode, O. H., Abdulganiyu & Taha Ait Tchakoucht. Modified genetic algorithm and fine-tuned long short-term memory network for intrusion detection in the internet of things networks with edge capabilities. *Appl. Soft Comput.* **155**, 111434 (2024).
18. Saheed, Yakub Kayode, Sanjay Misra, and Sabarathinam Chockalingam. Autoencoder via DCNN and LSTM models for intrusion detection in industrial control systems of critical infrastructures. *In 2023 IEEE/ACM 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, (IEEE, 2023)
19. Saheed, Y. K. & Chukwuere, J. E. Xaiensembletl-iov: A new explainable artificial intelligence ensemble transfer learning for zero-day botnet attack detection in the internet of vehicles. *Res. Eng.* **24**, 103171 (2024).
20. Nazir, A. et al. Advancing IoT security: A systematic review of machine learning approaches for the detection of IoT botnets. *J. King Saud Univ.-Comput. Inform. Sci.* **35**(10), 101820 (2023).
21. Neto, E.C.P.; Dadkhah, S.; Ferreira, R.; Zohourian, A.; Lu, R.; Ghorbani, A.A. CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. Sensors **2023**, 23, 13. Available from: https://doi.org/10.3390/s23135941.
22. Jony & Akinul Islam; Arjun Kumar Bose Arnob. A long short-term memory-based approach for detecting cyber-attacks in IoT using CIC-IoT2023 dataset. *J. Edge Comput.* **3** (1), 28–42 (2024).
23. Kasongo, S. M. & Sun, Y. Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *J. Big Data* **7**, 1–20 (2020).
24. Kunang, Y. N., Nurmaini, S., Stiawan, D. & Suprapto, B. Y. Attack classification of an intrusion detection system using deep learning and hyperparameter optimization. *J. Inf. Secur. Appl.* **58**, 102804 (2021).
25. Abdulhammed, R.; Faezipour, M.; Musafer, H.; Abuzneid,A. Efficient network intrusion detection using pca-based dimensionality reduction of features. *International Symposium on Networks, Computers and Communications (ISNCC)*, (IEEE, **2019)**
26. Bhattacharya, S. et al. A novel PCA-firefly based xgboost classification model for intrusion detection in networks using GPU. *Electronics* **9**(2), 219 (2020).
27. Shahid, B. et al. Enhancing IoT network security through deep learning-powered Intrusion Detection System. *Internet of Things* **24**, 100936 (2023).
28. Sharma, B., Sharma, L., Lal, C. & Roy, S. Anomaly based network intrusion detection for IoT attacks using deep learning technique. *Comput. Electr. Eng.* **107**, 108626 (2023).

Acknowledgements

The author extends his appreciation to the Deanship of Scientific Research at Northern Border University, Arar, KSA, for funding this research work through the project number "NBU-FFR-2025-1576-04".

Author contributions

Author-1: Abdullah Mujawib - Conceptualization, Investigation, design, preparing diagram and tables, analysis the resultsAuthor-2: Fahad Alqahtani - Interpretation, Writing original draft preparation and writing, editing and proof reading.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to A.M.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025

Content courtesy of Springer Nature, terms of use apply. Rights reserved

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").
Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.
These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.
We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.
While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.
These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.
To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.
Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.
If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com